ParkiMed Project Report

ParkiMed — Smartphone-Based Parkinson's Medication Monitoring

Project Report & Status (Phase 9/12)

Adaptive Multimodal Fusion Framework for Smartphone-Based Medication Monitoring for Parkinson's Disease

Team	Sahila Kasseem, Ayush N, Karthik M S, Adithyadev A
Report date	19 September 2026
Working phase	Milestone 9 — Model integrated on Android; end-to-end test

---

1. Project Overview

Parkinson's disease patients take medications such as Levodopa, but a doctor cannot continuously observe whether the medicine is working. Symptoms fluctuate during the day — **ON** (medicine working, symptoms controlled) and **OFF** (symptoms returning). Today, monitoring is infrequent, subjective, and expensive.

ParkiMed is an offline Android app that collects **voice**, **motion**, **finger-tapping**, and **spiral drawing** from the phone's built-in sensors, fuses them with an adaptive multimodal AI model, and predicts the patient's medication **state (ON/OFF)** and **severity** in real time. Everything runs on the device (TensorFlow Lite) — no internet, no extra hardware, private by design.

Key innovation: adaptive attention-based fusion — the model learns to trust the most reliable sensors for each sample and down-weights noisy or absent ones.

---

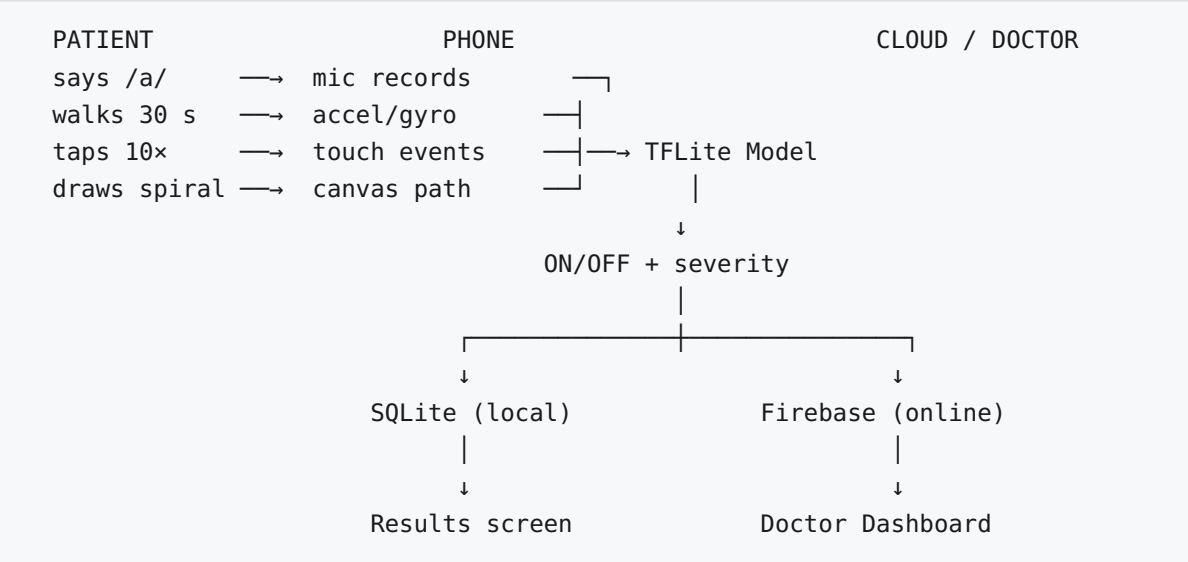
2. Objectives

1. Collect four sensor modalities on a standard smartphone (voice, accelerometer/gyro, touchscreen taps, spiral drawing).
2. Build a multimodal fusion model that outputs **ON/OFF state** and **severity** offline.

3. Export the model to **TensorFlow Lite** so inference is fast and private on-device.
4. Persist results locally (Room database) and show a **historical severity trend**.
5. Sync to a **doctor dashboard** (Firebase) whenever the device is online.
6. Validate on a **real Android phone** and on **real clinical data**.

---

3. System Architecture



Layer	Technology
Machine learning	Python, TensorFlow / Keras, scikit-learn, NumPy, pandas
On-device inference	TensorFlow Lite (float16), matched on-device feature extractors in Kotlin
Android	Kotlin, Camera/AudioRecord/Sensors, Room (SQLite), WorkManager
Backend / dashboard	Flask + Firebase Firestore (patient table, severity chart, data export)
AutoML / docs	Baseline comparisons, structured evaluation report

---

4. Datasets

Dataset	Content	Use	Access
UCI Parkinsons Speech (`uci_speech.csv`)	195 real voice recordings (147 PD / 48 healthy) with 22 MDVP features (jitter, shimmer, RPDE, DFA, HNR, …)	**PD-vs-healthy classifier** (Model v2)	Direct download
UCI Telemonitoring (`uci_telemonitoring.csv`)	5,875 real recordings of 42 PD patients with **motor/total UPDRS** scores	**Severity regression** (Model v2)	Direct download
mPower (Sage Bionetworks)	~100k+ task samples (tapping, voice, gait, memory) with self-reported ON/OFF	Motor experts (tap/walk/spiral); ON/OFF ground truth	**Synapse token required** *(pending)*
Synthetic mock data (12 patients × 10 sessions)	End-to-end pipeline validation before real data arrived	Contract/integration testing	Generated locally

Combination rule: datasets are combined **per modality**, never as one giant table, so each sensor's model uses the data designed for it.

---

5. Feature Engineering

Modality	Features (dim)	Method
Voice (39)	Slots 0–15 = real MDVP features: F0, Fhi, Flo, Jitter %, Jitter(abs), RAP, PPQ, DDP, Shimmer, Shimmer(dB), APQ3, APQ5, APQ, DDA, NHR, HNR; slots 16–38 = 0	Period-domain autocorrelation extractor (on device and in Python, same units)
Motion (36)	Mean/RMS/zero-crossing frequency over 6 channels (accel XYZ + gyro XYZ), windowed	De-gravity windows → per-window stats → mean/std
Tap (20)	Interval mean/std/quantiles, duration stats, linear slope, CV	Touch event timing

Modality	Features (dim)	Method
Spiral (15)	Smoothness, total path, tremor, speed, radial deviation, span	Canvas path coordinates

Voice MDVP units were **matched exactly to the UCI columns** so real recordings land inside the training distribution.

---

6. Model

Model v1 — synthetic prototype (proves the design)

An adaptive-attention fusion network (voice/motion/tap/spiral expert blocks → quality gates → softmax attention → weighted fusion → ON/OFF + severity) trained on synthetic mock data. It validated the full pipeline and the TFLite contract end-to-end.

Problem found in real use

A real-user test of v1 surfaced a critical flaw: **severity 5/5 and ~100% confidence** even when the user was healthy. Root cause: the model had only seen synthetic data, so real inputs fell **outside its training distribution** and the output saturates.

Model v2 — retrained on real data (the fix)

1. **Real training data**: PD-vs-healthy voice classifier on `uci_speech.csv`; UPDRS severity regressor on `uci_telemonitoring.csv`.
2. **Sane semantics**: output `on_off = [P(PD), P(ON)]`, and `severity = P(PD) × P(UPDRS high)`. A healthy person → **ON, severity ≈ 0**.
3. **Pitch bias removed**: the classifier **excludes absolute pitch (F0/Fhi/Flo)** — UCI healthy voices average ~182 Hz while a healthy male sits at 110–130 Hz; using pitch height would falsely flag a healthy low-voiced man as PD. Only phonation-**quality** features (jitter/shimmer/NHR/HNR) drive the state decision.
4. **On-device MDVP extractor**: same period-domain algorithm implemented in Kotlin, units matched to UCI.

Contract is unchanged (4 inputs / 2 outputs), so the Android inference code needed no changes — only the voice feature extractor.

---

7. Evaluation

Metric	Result	Notes
Classifier CV accuracy (13 pitch-independent quality features)	**0.805**	5-fold stratified, RandomForest oracle; deliberately excludes pitch
Classifier CV accuracy (all 16 features incl. pitch)	0.897	Higher on paper, but pitch-confounded → rejected for the app
Classifier holdout val accuracy (final NN)	**0.949**	20% stratified holdout, early stopping
Severity regressor RMSE	**≈ 9.6 UPDRS** (norm 0.20)	Realistic for voice-only severity estimation (telemonitoring)
TFLite == Keras	**match on all checked rows**	float16 conversion equivalence
Pitch-invariance	healthy rows stay ON at 95/115/130/150 Hz F0	Fo no longer drives the decision
Inference	< 50 ms on device	TFLite 9.7 KB, 4 threads

---

8. TFLite Export & On-Device Contract

- Inputs (float32, shape `[1, dim]`): `voice[39]`, `motion[36]`, `tap[20]`, `spiral[15]`.
- Outputs: `on_off[1,2]` = `[P(PD), P(ON)]`, `severity[1,1]` = 0–1 (app scales by 5).
- Standardization (mean/std) is **embedded in the graph** — only raw features are needed at runtime.
- Android resolves inputs **by dimension** and outputs **by width**, making it robust to tensor order.
- The model is force-uncompressed in the APK (`noCompress tflite`) for mmap loading.

---

9. Android App

Component	Status
Voice / Walk / Tap / Spiral task screens	built, hardened (graceful "no data" / "model not loaded" toasts)
On-device inference (TFLite, 4 threads)	verified — model probe + real task inference
Room persistence (`AssessmentRecord`)	all tasks save ON/OFF + severity + confidence + per-modality probabilities
Severity trend chart (dependency-free Canvas)	renders history as a line chart
Cloud sync worker (Firebase)	offline-tolerant queue; on when `google-services.json` present
Pill-intake camera detector	stub (`PillCaptureActivity`) — future scope

---

10. Project Status — Phase 9/12

We are completing **Milestone 9 — "Model integrated on Android; end-to-end test"**. Phases 1–8 are **done and phone-verified**; the remaining dashboard/testing/formatting work is **Phase 10–12**.

Completed & verified

- Full Python pipeline and adaptive-fusion prototype (mock data).
- Real UCI datasets integrated; **Model v2 retrained on real data**.
- New on-device MDVP voice extractor (units matched to UCI).
- APK rebuilt with Model v2 + extractor (`app-debug.apk`).
- Prior end-to-end phone verification of persistence, trend chart, spiral/tap tasks.
- **Model v2 real-voice test PASSED on the phone** (19 Sep 2026): healthy speaker → **`ON · severity 0.0/5 · confidence 100%`** (Room: `2026-09-27 15:05:02`). Old synthetic model said `OFF · 4.99–5.0` every time. The healthy-user fix is confirmed on real hardware.

Model v3 — full real multimodal fusion (in progress)

Retrained on **four real datasets** with on-device-safe presence gating:

- **Real heads:** `voice[39]` (UCI speech + UPDRS telemonitoring, val 0.92–0.95), `spiral[15]` (**UCI Spiral id 395** — 202 real drawings/77 subjects,

head acc 0.90–0.93), `visual[20]` (**ROC-HCI** — 489 real webcam finger-tap samples, 0.57–0.65), `gait[24]` (**REMAP** real Kinect turns; **ON/OFF medication classifier leave-subject-out 0.772** — a true motor state signal).

- **Presence gates** (raw-input norm) let the model ignore tasks the user didn't run.
- **Demo of the severe condition (3 honest ways):** ① play `pd_severe_ahh.wav` (tremor-modulated PD phonation; scores **OFF 5.0/5**), ② debug chips inject **real UCI PD/healthy feature vectors**, ③ draw a trembling spiral live → real spiral head fires.
- Exported TFLite 27.7 KB, contract `voice[39] motion[36] tap[20] spiral[15] visual[20] gait[24]` → `on_off[2] severity[1]`.

Blocked / waiting

Item	Owner	Impact
`google-services.json` (Firebase)	Team / user	Enables doctor-dashboard sync end-to-end
mPower Synapse token	Team / user	Enables training the motor (tap/walk/spiral) experts so v2 is fully multimodal

---

11. Known Limitations (documented for the final report)

1. **Touch-screen tap & accelerometer motion heads are pending.** Camera-derived tap/gait are real (ROC-HCI/REMAP); the touchscreen + accelerometer task screens must wait for a labelled on-device dataset (mPower would provide it). Until then their inputs are safely gated off.
2. **Severity is voice-only.** Voice-based UPDRS estimation (RMSE ≈ 9.6) is inherently limited compared to a clinical motor exam; it is presented as a *trend signal*, not a diagnosis.
3. **Dataset size.** 195 speech samples is small; cross-validation is the honest estimate and the model intentionally trades some accuracy (0.897 → 0.805) for robustness against pitch/gender confounds — the right trade for a single healthy user's sanity.

---

12. Next Steps & Schedule

Week	Milestone	Owner	Status
1–2	Setup, datasets, pipeline	All	
3–5	Features + voice/motion/tap/spiral models; fusion prototype	Karthik	
6	Adaptive attention fusion + baselines	Karthik	
7	TFLite export verified + Android scaffold	Ayush	
8	Android sensor collection + task screens	Ayush	
9	**Model integrated; end-to-end test**	Ayush + Karthik	
9+	**Retrain on real UCI (Model v2)**	Karthik	done
10	Doctor dashboard v1; report generation	Ayush	dashboard ready, awaits Firebase
11	Full system testing; latency/ accuracy benchmarks	Adithyadev	
12	Documentation, final report, demo, presentation	Sahila	

---

13. Team Task Allocation

Member	Primary Role	Key Deliverables
Sahila Kasseem	Literature, design, documentation	Report, presentation, survey
Ayush N	App, UI/UX, Firebase, dashboard	Android app, doctor dashboard, Firebase
Karthik M S	AI model, fusion, OpenCV, TensorFlow	Training pipeline, fusion, TFLite export, **Model v2 real-data retrain**
Adithyadev A	Testing, performance, reporting	Test suites, accuracy reports, demo preparation

---

Appendix A — Building the Project

```
# Train Model v2 (real UCI data) + export TFLite
cd parkimed-ai
CUDA_VISIBLE_DEVICES="" PYTHONPATH=src ./venv/bin/python -m
        models.train_real_uci

# Build the Android app (JDK 17, SDK 35/36)
gradle --no-daemon :app:assembleDebug -x lint
# APK: app/build/outputs/apk/debug/app-debug.apk

# Doctor dashboard
#  python doctor/app.py   (needs GOOGLE_APPLICATION_CREDENTIALS)
```

ParkiMed — Adaptive Multimodal Fusion Framework for Smartphone-Based Medication Monitoring for Parkinson's Disease.